

NixOS Android Builder

Ephemeral, Attested AOSP Build System

TII

Contents

Preface	7
Quick Start	7

I	User Guide	
1	Getting Started	11
1.1	Requirements	11
1.2	Generate Secure Boot Keys	11
2	Building & Signing Images	13
2.1	Build	13
2.2	Sign	13
2.3	Configure Attestation Server	13
2.4	Flash	13
3	Disk Installer	15
3.1	Build & Sign	15
3.2	Configure Install Target	15
3.3	Verify Configuration	15
3.4	Test in a VM	15
4	Running Android Builds	17
4.1	Fetch Source	17
4.2	Build	17
4.3	SBOM Generation	17
4.4	Source Measurement	17
4.5	Save Build Outputs	17
4.6	Credential Storage	18
4.7	Inspect PCR State	18
4.8	Reset to Initial State	18
4.9	Updates	18
5	Desktop Variant	19
5.1	Prerequisites	19
5.2	Build	19
5.3	Customise	19
5.4	Sign & Flash	19
5.5	Prepare Target UEFI	19

5.6	Install	19
5.7	First Boot	20
5.8	Verify	20
6	Virtual Machine Testing	21
6.1	Builder VM	21
6.2	Installer VM	21
6.3	Exiting	21

II

Attestation

7	Attestation Server Setup	25
7.1	What It Runs	25
7.2	System Users & File Locations	25
7.3	Module Structure	25
7.4	Prerequisites	26
7.5	Deploy	26
7.6	Verify	26
7.7	Open Firewall	26
7.8	Distribute CA Certificate	26
7.9	Update	26
7.10	Deactivate	26
8	Attestation Protocol	27
8.1	Architecture	27
8.2	Ports & Protocols	27
8.3	Data Exchanged	28
8.4	Full Boot & Attestation Sequence	28
8.5	Measured Boot Reference State	28
8.6	PCR Validation Policy	29
8.7	TLS / PKI	29
8.8	Agent Configuration	29
9	Attestation-Gated Git Server	31
9.1	How It Works	31
9.2	Managing Repositories	32
9.3	Cloning from an Agent	32
9.4	Verifying the Gate	32
9.5	Nix Options	33

III

Architecture

10	Design Principles	37
10.1	Limitations	37
10.2	Technologies	37
10.3	Deployment Topology	37
11	Build Process	39
11.1	Steps	40
11.2	Disk Layout	41
12	Boot Sequence	43
12.1	Steps	45
13	Components	47
13.1	Custom FHS Environment	47
13.2	Android Build Environment	47
13.3	YubiKey Authentication	47
13.4	Unattended Mode	47
13.5	Remote Attestation (Keylime)	47
13.5.1	Auto-Enrollment	47
13.5.2	Attestation Policy	48
13.5.3	Tools	48
13.6	Credential Storage	48
13.7	Fatal Error Handling	49
13.8	Debug Mode	49

IV

Reference

14	NixOS Options Reference	53
15	Troubleshooting	59
15.1	Secure Boot	59
15.2	Boot	59
15.3	Android Build	59
15.4	Attestation	59
15.5	VM Testing	60
16	Glossary	61

V

Appendices

17	Demo Runbook: Attestation-Gated Git	67
17.1	What We're Showing	67
17.2	Pre-Demo Prep	67
17.2.1	1. Populate demo repository on the server	67
17.2.2	2. Verify agent credentials	67

17.2.3	3. Terminal layout	67
17.2.4	4. Checklist (30 min before)	67
17.3	Live Script	68
17.3.1	Act 1 — The problem (1 min, verbal)	68
17.3.2	Act 2 — Attestation status (2 min)	68
17.3.3	Act 3 — Attested clone (2 min)	68
17.3.4	Act 4 — Revocation (3 min)	68
17.3.5	Act 5 — Q&A (2 min)	68
17.4	Recovery	68
18	Contributing	69
18.1	Quick Reference	69
18.2	Development Workflow	69
18.3	Tests	69
18.4	Project Structure	69
18.5	Security Notes	70

Preface

The NixOS Android Builder produces signed, ephemeral disk images for building AOSP. Each boot starts clean: the build partition is encrypted with a fresh random key and wiped on shutdown. A Keylime attestation server continuously verifies that builder machines are running unmodified software.

Quick Start

```
nix run .#create-signing-keys      # generate Secure Boot keys
nix build .#image                  # build the disk image
install -m 600 result/*.raw .      # copy out of the nix store
nix run .#configure-disk-image -- sign --keystore ./keys --device *.raw
sudo dd bs=1M status=progress if=*.raw of=/dev/sdX && sudo sync
```

Or try it in a VM:

```
nix run .#run-vm
```




User Guide

1	Getting Started	11
1.1	Requirements	11
1.2	Generate Secure Boot Keys	11
2	Building & Signing Images	13
2.1	Build	13
2.2	Sign	13
2.3	Configure Attestation Server	13
2.4	Flash	13
3	Disk Installer	15
3.1	Build & Sign	15
3.2	Configure Install Target	15
3.3	Verify Configuration	15
3.4	Test in a VM	15
4	Running Android Builds	17
4.1	Fetch Source	17
4.2	Build	17
4.3	SBOM Generation	17
4.4	Source Measurement	17
4.5	Save Build Outputs	17
4.6	Credential Storage	18
4.7	Inspect PCR State	18
4.8	Reset to Initial State	18
4.9	Updates	18
5	Desktop Variant	19
5.1	Prerequisites	19
5.2	Build	19
5.3	Customise	19
5.4	Sign & Flash	19
5.5	Prepare Target UEFI	19
5.6	Install	19
5.7	First Boot	20
5.8	Verify	20
6	Virtual Machine Testing	21
6.1	Builder VM	21
6.2	Installer VM	21
6.3	Exiting	21

1. Getting Started

1.1 Requirements

- A **local machine** with [Nix](#) installed, building for `x86_64-linux`.
 - ~4 GB memory for evaluation
 - ~30 GB disk space for dependencies and images
- A **USB mass storage** device (~3 GB) to transfer the image.
- A **target machine** for building Android:
 - ~64 GB memory
 - ~250 GB free disk space
 - UEFI with Secure Boot in Setup Mode
- A **git checkout** of this repository. Commands assume the repository root unless stated otherwise.
- *(Optional)* An **attestation server** running Keyline. See Chapter 7.. Not required for Chapter 6..

1.2 Generate Secure Boot Keys

The `create-signing-keys` helper generates a PK/KEK/DB key set in `keys/`:

```
nix run .#create-signing-keys
```

These keys are required for Secure Boot signing and enrollment. They **must not** be committed plain-text. Tools such as [SOPS](#) can manage them securely.

2. Building & Signing Images

2.1 Build

Build the disk image. Dependencies are fetched from cache.nixos.org; subsequent runs are cached.

```
nix build --print-build-logs .#image
```

💡 Tip

[nix-output-monitor](#) gives better build progress. Install with `nix shell nixpkgs#nom`, then use `nom build` instead.

The result is a raw disk image (~2.7 GB) with a full partition table and NixOS closure.

2.2 Sign

`configure-disk-image sign` signs the UKI and copies Secure Boot enrollment bundles. It runs outside the Nix sandbox to avoid leaking keys into `/nix/store`.

Copy the image out of the read-only store, then sign:

```
install -m 600 result/*.raw .  
nix run .#configure-disk-image -- sign --keystore ./keys --device *.raw
```

2.3 Configure Attestation Server

The image needs to know how to reach the attestation server. This writes the server address and CA certificate to the ESP:

```
nix run .#configure-disk-image -- set-attestation-server \  
  --ip 10.0.0.1 \  
  --ca-cert /path/to/ca-cert.pem \  
  --device *.raw
```

If skipped, the keylime agent will fail to start on boot. See Section 8.8 for the JSON schema.

See Chapter 7. for deploying the server itself.

2.4 Flash

Flash the signed image to a USB stick or external drive:

```
sudo dd bs=1M status=progress if=*.raw of=/dev/sdX  
sudo sync
```

Use `lsblk` to identify the target device.

Enable Secure Boot Setup Mode in the target machine's firmware before first boot. This is needed for automatic key enrollment. After enrollment, Setup Mode is not needed again unless keys are rotated.

3. Disk Installer

The installer image contains a minimal ESP plus the builder image as payload. It boots and flashes the payload to a local disk.

3.1 Build & Sign

```
nix build -L .#installer-image
install -m 600 result/disk-installer.raw .
nix run .#configure-disk-image -- sign --keystore ./keys --device disk-
installer.raw
```

The `sign` sub-command accepts flags:

- `--installer` — sign only the installer UKI
- `--payload` — sign only the payload UKI
- `--no-auto-enroll` — skip copying Secure Boot enrollment bundles

3.2 Configure Install Target

Interactive (user selects disk at boot):

```
nix run .#configure-disk-image -- set-target --target select --device disk-
installer.raw
```

Automatic (no prompt):

```
nix run .#configure-disk-image -- set-target --target /dev/vda --device disk-
installer.raw
```

3.3 Verify Configuration

```
nix run .#configure-disk-image -- status --keystore ./keys --device disk-
installer.raw
```

3.4 Test in a VM

```
nix run .#installer-vm
```


4. Running Android Builds

After booting, you get a shell with the Android build toolchain.

The scripts below (`fetch-android`, `build-android`, etc.) are reference implementations shipped as convenience wrappers. They demonstrate the intended workflow but can be replaced or extended for specific distribution needs.

4.1 Fetch Source

```
fetch-android
```

Uses `repo` to clone AOSP to `/var/lib/build/source`. If multiple branches are configured via `nixosAndroidBuilder.build.branches`, `select-branch` presents a menu on boot.

Run `fetch-android --help` for flags (manifest URL, branch, source directory, git identity).

4.2 Build

```
build-android
```

Run `build-android --help` for flags (lunch target, source directory).

Lower-level tools (`lunch`, `m`, `ninja`) are available after loading Android's `envsetup.sh`:

```
cd /var/lib/build/source
source build/envsetup.sh
```

See [upstream documentation](#) for details.

4.3 SBOM Generation

```
android-sbom
```

Generates a Software Bill of Materials in `/var/lib/build/source/out/soong/sbom` using [upstream SBOM facilities](#).

4.4 Source Measurement

```
android-measure-source
```

Hashes all files across all git repositories in the checkout, producing `out/source_measurement.txt`.

4.5 Save Build Outputs

Outputs are in `/var/lib/build/source/out` — images in `out/target/product`, logs in `error.log` and `verbose.log.gz`.

Since `/var/lib/build` is ephemeral, outputs are lost on shutdown. To persist them, set `nixosAndroidBuilder.artifactStorage.enable = true`. During boot, a second disk is selected for persistent storage at `/var/lib/artifacts`. It can also be pre-configured:

```
nix run .#configure-disk-image -- set-storage --target /dev/sdb --device *.raw
```

The `copy-android-outputs` script copies outputs matching `nixosAndroidBuilder.artifactStorage.contents` to `/var/lib/artifacts`.

4.6 Credential Storage

A TPM-backed credential store persists secrets across reboots (see Section 13.6).

```
echo 'my-secret-token' | credential-store add api-token
credential-store list
credential-store show api-token
credential-store remove api-token
```

Credentials can also be added from a file:

```
credential-store add api-token ~/token.txt
```

Credentials are bound to PCR 7 (Secure Boot policy) by default. See Chapter 14. for customization (e.g. adding PCR 11 binding).

4.7 Inspect PCR State

TPM PCR values can be read from sysfs:

```
cat /sys/class/tpm/tpm0/pcr-sha256/7
```

On boot, `report-measured-boot-state` sends the measured boot reference state to the attestation server automatically. For debugging:

`measure-boot-state` generates the reference state from the UEFI event log:

```
measure-boot-state -o /tmp/refstate.json
```

`debug-measured-boot-state` diagnoses attestation failures:

```
debug-measured-boot-state save                # before reboot
debug-measured-boot-state                    # after reboot – auto-
detects saved state
debug-measured-boot-state diagnose -r enrolled.json # compare against
enrolled state
```

4.8 Reset to Initial State

Reboot. The ephemeral encryption key is lost, rendering all build data inaccessible. The `/var/lib/build` partition is reformatted on next boot.

```
systemctl reboot
```

4.9 Updates

Update all dependencies by updating the pinned nixpkgs:

```
nix flake update
```

Rebuild the image and run tests (see Chapter 18.) to catch regressions.

5. Desktop Variant

A persistent NixOS workstation with optional GNOME, intended for physical hardware.

5.1 Prerequisites

- ☐ Nix installed on your local machine
- ☐ Git checkout of this repository
- ☐ USB stick (≥ 4 GB)
- ☐ Target machine with UEFI, Secure Boot support, and a disk to install to

5.2 Build

Shell (minimal) desktop:

```
nix build -L .#desktop-installer-image
install -m 600 result/disk-installer.raw desktop-installer.raw
```

GNOME desktop:

```
nix build -L .#desktop-gnome-installer-image
install -m 600 result/disk-installer.raw desktop-installer.raw
```

5.3 Customise

Edit `desktop-configuration.nix`:

- ☐ Set YubiKey public keys in `yubikeys.groupA` (or clear for password auth)
- ☐ Set `nixosAndroidBuilder.debug = false` for production

5.4 Sign & Flash

```
nix run .#configure-disk-image -- sign --keystore ./keys --device desktop-
installer.raw
nix run .#configure-disk-image -- status --keystore ./keys --device desktop-
installer.raw
sudo dd bs=1M status=progress if=desktop-installer.raw of=/dev/sdX
sudo sync
```

- ☐ Both “Installer” and “Payload” show “✓ Signed and verified”

5.5 Prepare Target UEFI

- ☐ Enable Secure Boot
- ☐ Put Secure Boot into **Setup Mode** (clear existing keys)
- ☐ Set USB as first boot device
- ☐ Save and reboot

5.6 Install

- ☐ Boot from USB — installer appears on `tty2`

- ☐ Select target disk (or auto-installs if pre-configured)
- ☐ Wait for copy to complete
- ☐ Remove USB when prompted, press Enter to reboot

5.7 First Boot

Two automatic reboots:

1. **Boot 1:** Secure Boot keys auto-enroll → automatic reboot
2. **Boot 2:** `systemd-repart` grows root partition → normal boot

5.8 Verify

- ☐ `tuigreet` login screen appears
- ☐ `bootctl status` shows `Secure Boot: enabled (user)`
- ☐ `df -h /` shows root using the full disk
- ☐ Network is up (`ip addr`)
- ☐ Data persists across `systemctl reboot`

6. Virtual Machine Testing

Test the full setup in QEMU. Requires KVM (enable VT-x/AMD-V in firmware; verify with `lsmod | grep kvm`).

6.1 Builder VM

```
nix run .#run-vm
```

Creates a writable image copy, signs it with test keys, and starts QEMU with Secure Boot and TPM. If `attestation-server.json` is present in the current directory, the keylime agent is configured from it (same format as `/boot/attestation-server.json`, see Section 8.8).

Subsequent runs reuse the existing `.raw` file. Delete it to start from a clean image.

6.2 Installer VM

```
nix run .#installer-vm
```

Tests the Chapter 3. workflow in a VM.

6.3 Exiting

```
systemctl poweroff # from within the VM
```

Or close the QEMU window.

II

Attestation

7	Attestation Server Setup	25
7.1	What It Runs	25
7.2	System Users & File Locations	25
7.3	Module Structure	25
7.4	Prerequisites	26
7.5	Deploy	26
7.6	Verify	26
7.7	Open Firewall	26
7.8	Distribute CA Certificate	26
7.9	Update	26
7.10	Deactivate	26
8	Attestation Protocol	27
8.1	Architecture	27
8.2	Ports & Protocols	27
8.3	Data Exchanged	28
8.4	Full Boot & Attestation Sequence	28
8.5	Measured Boot Reference State	28
8.6	PCR Validation Policy	29
8.7	TLS / PKI	29
8.8	Agent Configuration	29
9	Attestation-Gated Git Server	31
9.1	How It Works	31
9.2	Managing Repositories	32
9.3	Cloning from an Agent	32
9.4	Verifying the Gate	32
9.5	Nix Options	33

7. Attestation Server Setup

The `system-manager/` directory turns a stock Linux machine (Ubuntu, Debian, etc.) into a Keylime attestation server — registrar, verifier, auto-enrollment daemon, and an optional attestation-gated git server.

`System-manager` applies NixOS-style module configurations to non-NixOS hosts without replacing the OS.

7.1 What It Runs

- **keylime-registrar** — Accepts agent registrations (EK + AK identity)
- **keylime-verifier** — Continuously attests enrolled agents via TPM quotes
- **keylime-auto-enroll** — Bridges registration and verification; auto-enrolls agents
- **keylime-tls** — Oneshot service; auto-generates a self-signed CA and mTLS PKI
- **keylime-git-nginx** / **keylime-git-auth** (*optional*) — Attestation-gated git server (see Chapter 9.)

See Chapter 8. for ports, data flow, and PKI details.

7.2 System Users & File Locations

- `keylime` user — runs all services, owns `/var/lib/keylime`
- `tss` group — TPM device access (`/dev/tpmrm0`)
- `/var/lib/keylime/` — Keylime state (SQLite databases)
- `/var/lib/keylime/tls/` — Auto-generated TLS PKI
- `/etc/keylime/` — Configuration files (`registrar.conf`, `verifier.conf`, `tenant.conf`, `ca.conf`, `logging.conf`)

7.3 Module Structure

Two Nix modules in `system-manager/`:

- `tpm2.nix` — udev rules and environment variables for TPM access (host kernel must have TPM support)
- `keylime.nix` — all Keylime services; shared helpers from `modules/lib/keylime-shared.nix`

Wired in `flake.nix`:

```
systemConfigs.default = system-manager.lib.makeSystemConfig {
  modules = [
    ./system-manager/tpm2.nix
    ./system-manager/keylime.nix
  ]
  {
    nixpkgs.hostPlatform = "x86_64-linux";
    services.keylime = {
      enable = true;
      registrar.enable = true;
      verifier.enable = true;
      autoEnroll.enable = true;
    };
  }
];
```

7.4 Prerequisites

- A **local machine** with Nix installed ([flakes enabled](#))
- A **target Linux x86_64** machine reachable via SSH as **root**
- **Network access** from agents to the target on ports 8891, 8881, 8893

Nix does not need to be installed on the target.

7.5 Deploy

```
git clone <repository-url> nixos-android-builder
cd nixos-android-builder
nix run 'github:numtide/system-manager' -- \
  --target-host root@<server-ip> switch --flake .#default
```

Activation creates the `keylime/tss` users, writes config to `/etc/keylime/`, generates the TLS PKI, and starts all services.

7.6 Verify

```
systemctl status keylime-tls keylime-registrar keylime-verifier keylime-auto-
enroll
ls -la /var/lib/keylime/tls/          # TLS certificates
curl -k https://127.0.0.1:8891/v2.5/agents/ # expect mTLS error – registrar
is up
```

7.7 Open Firewall

```
sudo ufw allow 8891/tcp # Registrar (mTLS)
sudo ufw allow 8881/tcp # Verifier (mTLS)
sudo ufw allow 8893/tcp # Auto-enroll (server TLS, TOFU)
sudo ufw allow 443/tcp  # Git server (mTLS, optional – only if
gitServer.enable is set)
```

7.8 Distribute CA Certificate

Copy `/var/lib/keylime/tls/ca-cert.pem` from the server. It is baked into agent images via `configure-disk-image set-attestation-server` (see Chapter 2.).

7.9 Update

```
git pull
nix run 'github:numtide/system-manager' -- \
  --target-host root@<server-ip> switch --flake .#default
```

7.10 Deactivate

On the target:

```
/nix/var/nix/profiles/system-manager/bin/deactivate
```

Stops managed services and removes config. Does not remove `/var/lib/keylime` state.

8. Attestation Protocol

Ports, data flow, PCR policy, and PKI reference.

8.1 Architecture

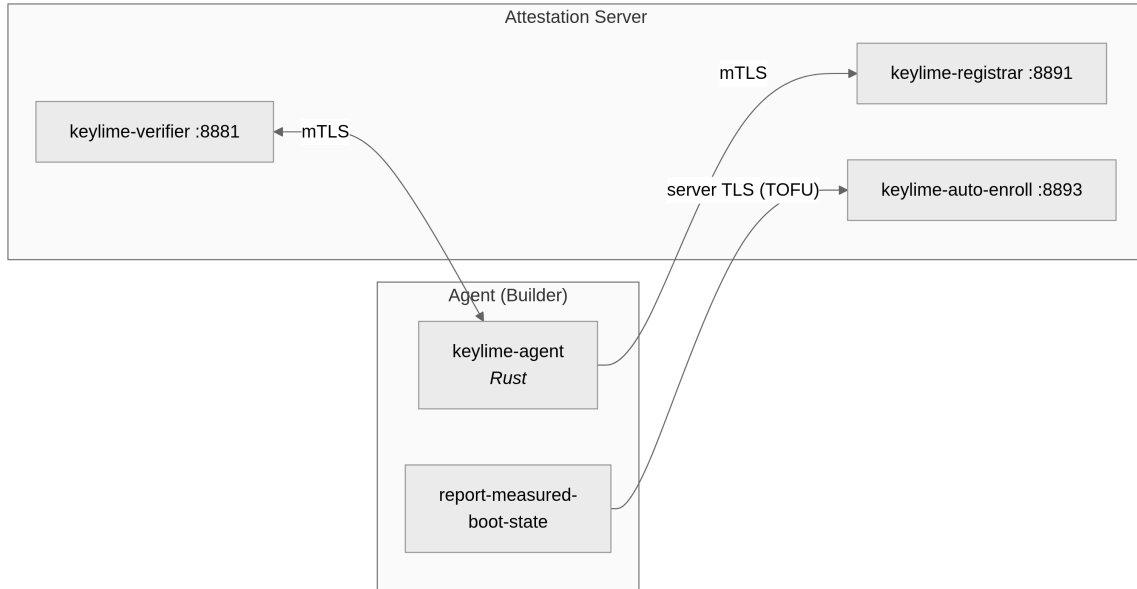


Figure 8.1: Attestation architecture

8.2 Ports & Protocols

Port	Service	Direction	Protocol	Purpose
8891	Registrar	Agent → Server	HTTPS (mTLS)	Agent registration (EK + AK identity)
8881	Verifier	Server ↔ Agent	HTTPS (mTLS)	TPM quote requests, enrollment via keylime_tenant
8893	Auto-enroll	Agent → Server	HTTPS (server TLS, TOFU)	Agent POSTs measured boot reference state
443	Git server (nginx)	Agent → Server	HTTPS (mTLS)	Attestation-gated git clone (optional)
8895	Git auth	localhost	HTTP	Auth subrequest from nginx to keylime-git-auth (internal)

8.3 Data Exchanged

Port 8891 — Registrar: Agent POSTs its EK, AK, and UUID (SHA-256 of EK) on first boot. Stored in SQLite.

Port 8881 — Verifier: Accepts enrollment (`keylime_tenant -c add` with `--mb_refstate` and `uki` policy). Periodically sends nonces, receives TPM quotes + event logs, replays through the policy.

Port 8893 — Auto-enroll: Agent POSTs `{"uuid": "...", "measured_boot_state": {...}}`. Daemon matches against registrar entries and auto-enrolls.

8.4 Full Boot & Attestation Sequence

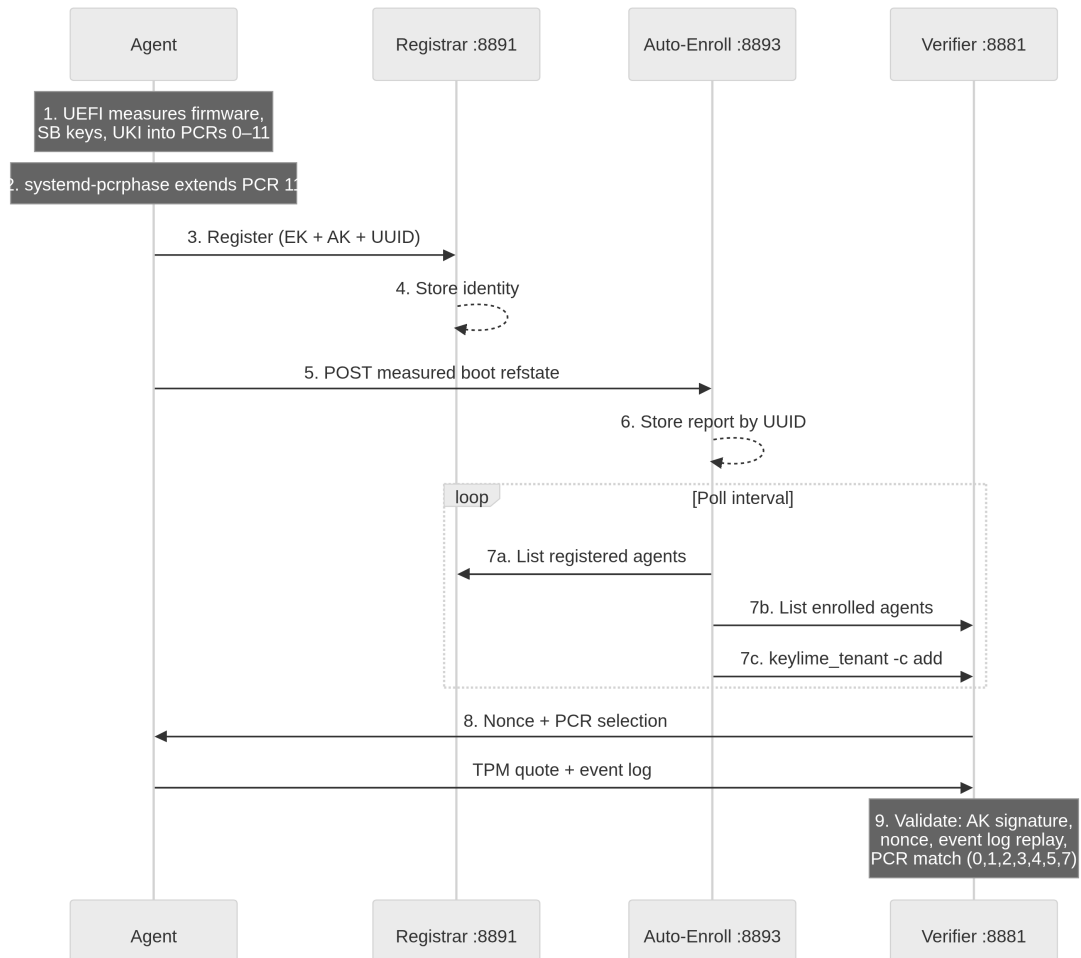


Figure 8.2: Boot and attestation sequence

8.5 Measured Boot Reference State

Generated by `measure-boot-state` from the UEFI binary event log:

Field	Source PCR	Pins
<code>scrtm_and_bios</code>	0	SCRTM version + firmware blob digests
<code>pk</code>	7	Platform Key certificate(s)
<code>kek</code>	7	Key Exchange Key certificate(s)
<code>db</code>	7	Allowed Signature Database entries

Field	Source PCR	Pins
<code>dbx</code>	7	Forbidden Signature Database entries
<code>uki_digest</code>	4	SHA-256 of the booted UKI binary

8.6 PCR Validation Policy

PCR	Content	Policy
0	SCRTM, firmware blobs	Pinned to refstate
1	BIOS settings, boot variables	Accepted
2	Boot services drivers	Accepted
3	Separator	Separator check
4	UKI application digest	Pinned to refstate
5	GPT partition table	Accepted
7	Secure Boot keys (PK, KEK, db, dbx)	Pinned to refstate
9	systemd-stub event tags	Accepted (tags vary per UKI; NvPCR runtime extension not in UEFI log)
11	UKI PE sections + pcphrase	Accepted (runtime extensions)

Pinned = event log digest must exactly match the enrolled reference state. **Accepted** = events are syntactically checked but any valid digest is allowed. **Separator check** = validates the EV_SEPARATOR event marking the boundary between pre-OS and OS-present measurements.

PCRs 9 and 11 are excluded from event-log-vs-TPM replay comparison (they receive runtime extensions not in the UEFI log) but are included in the TPM quote.

8.7 TLS / PKI

Auto-generated on the server by `keylime-tls.service`. Stored in `/var/lib/keylime/tls/`:

File	Used by	Purpose
<code>ca-cert.pem</code>	All	Root CA — distributed to agents
<code>server-cert.pem</code> / <code>server-key.pem</code>	Registrar, Verifier, Auto-enroll	Server identity
<code>client-cert.pem</code> / <code>client-key.pem</code>	Verifier, Auto-enroll, Tenant	Client identity for mTLS

8.8 Agent Configuration

`/boot/attestation-server.json` (written to ESP at build time):

```
{
  "ip": "10.0.0.1",
  "port": 8891,
  "verifier_port": 8881,
  "ca_cert": "-----BEGIN CERTIFICATE-----\n...\n-----END CERTIFICATE-----\n"
}
```

At boot, `keylime-configure-agent` reads this and patches `/run/keylime/agent.conf`.

9. Attestation-Gated Git Server

i Note

Demo feature. This git server is designed for demonstration purposes: it shows that only TPM-attested builder machines can clone sensitive repositories. It uses git dumb HTTP (static file serving) which is adequate for small, infrequently-updated repos but not suitable for large-scale production use.

The git server lets attested builder machines clone repositories over HTTPS/mTLS. A machine that fails attestation, or has never been attested, cannot clone.

9.1 How It Works

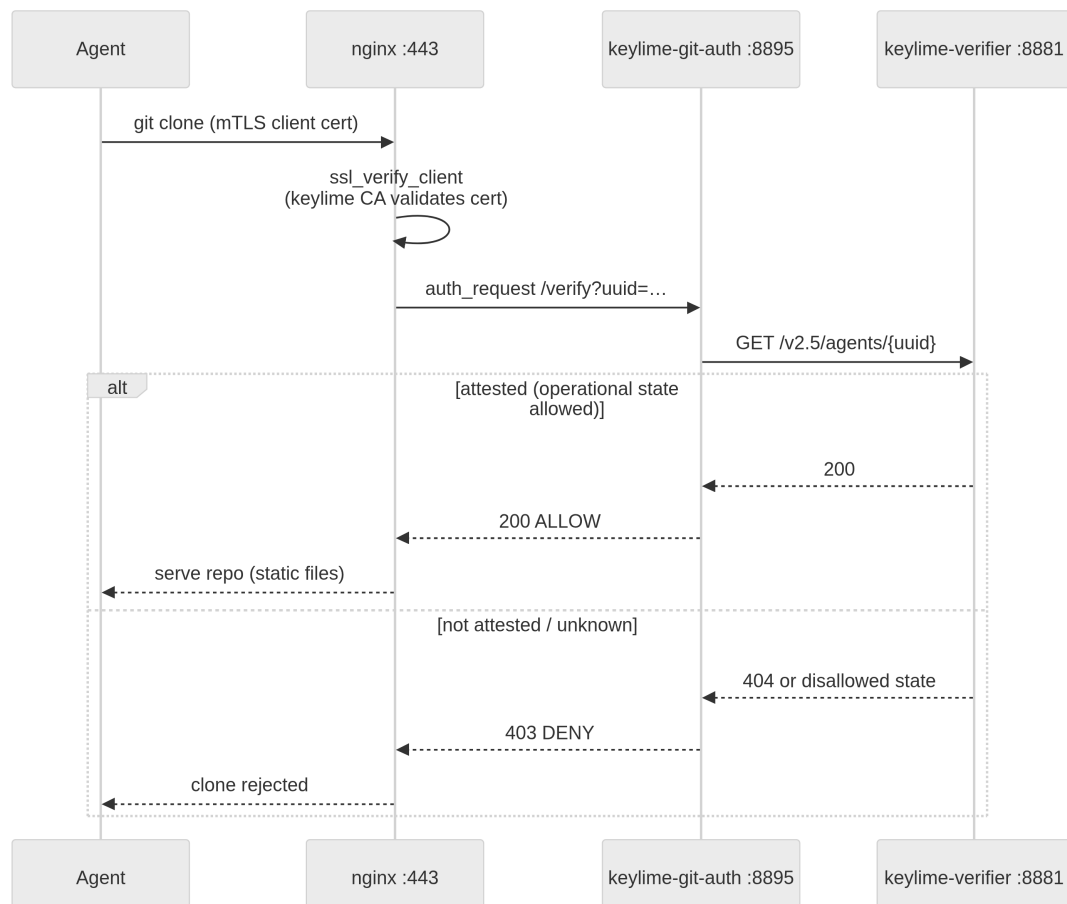


Figure 9.1: Git clone authentication flow

Identity. nginx enforces mTLS against the keylime CA. The agent UUID (SHA-256 of the TPM Endorsement Key) is the CN of the client certificate, so no separate credential is needed: a machine provisioned into keylime automatically has its git identity.

Authorization. `keylime-git-auth` is a small Python daemon on `127.0.0.1:8895`. It queries the keylime verifier for the agent's operational state and only allows requests from agents in a known-good

state (START, SAVED, GET_QUOTE, GET_QUOTE_RETRY, PROVIDE_V, PROVIDE_V_RETRY). Everything else is denied, including states it does not recognise. If the verifier is unreachable, it returns 503 and nginx rejects the clone (fail-closed).

Git transport. nginx serves the bare repo directory as static files (git dumb HTTP). No additional daemons are required. Run `git update-server-info` after each push, or install the provided post-receive hook, to keep the index files current.

9.2 Managing Repositories

Repositories are bare git repos in `/var/lib/keylime-git/repos/`.

Create a new repo:

```
git init --bare /var/lib/keylime-git/repos/my-repo.git
chown -R keylime:keylime /var/lib/keylime-git/repos/my-repo.git
git -C /var/lib/keylime-git/repos/my-repo.git update-server-info
ln -s /etc/keylime-git/hooks/post-receive \
    /var/lib/keylime-git/repos/my-repo.git/hooks/post-receive
```

Mirror an existing repo:

```
git clone --bare https://github.com/example/repo.git \
    /var/lib/keylime-git/repos/repo.git
chown -R keylime:keylime /var/lib/keylime-git/repos/repo.git
git -C /var/lib/keylime-git/repos/repo.git update-server-info
ln -s /etc/keylime-git/hooks/post-receive \
    /var/lib/keylime-git/repos/repo.git/hooks/post-receive
```

The post-receive hook runs `git update-server-info` automatically after every push so the index files stay current.

9.3 Cloning from an Agent

Builder machines include the `keylime-git-clone` wrapper, which automatically configures git with the machine's mTLS credentials:

```
keylime-git-clone https://<server-ip>/my-repo.git
```

This is equivalent to running `git clone` with the client certificate, key, and CA cert flags set. An optional destination directory can be appended:

```
keylime-git-clone https://<server-ip>/my-repo.git /path/to/dest
```

The credentials are provisioned automatically:

- **Client cert & key** are written to `/run/keylime-git/` by `report-measured-boot-state` after successful attestation.
- **CA cert** is written to `/run/keylime-git/ca-cert.pem` by `report-measured-boot-state` from `/boot/attestation-server.json`.

Both paths are readable by the `user` account so `keylime-git-clone` works without elevated privileges. The key is scoped to this machine's TPM identity and is only useful when paired with a passing attestation.

Custom cert paths can be passed if needed:

```
keylime-git-clone --cert=/path/to/cert --key=/path/to/key --ca=/path/to/ca \
    https://<server-ip>/my-repo.git
```

9.4 Verifying the Gate

Check both services are running:

```
systemctl status keylime-git-nginx keylime-git-auth
```


Confirm nginx rejects an unauthenticated request:

```
curl -k https://127.0.0.1:443/any-repo.git/info/refs
# Expected: 400 No required SSL certificate was sent
```

Watch the attestation gate in real time:

```
journalctl -fu keylime-git-auth
# Prints ALLOW / DENY with the agent UUID on every request
```

9.5 Nix Options

Option	Default	Description
services.keylime.gitServer.enable	services.enable	Enable the git server
services.keylime.gitServer.port	443	nginx HTTPS listen port
services.keylime.gitServer.authPort	8895	Auth daemon localhost port
services.keylime.gitServer.libpath	server/libkeylime-git/repos	Bare repo root

III

Architecture

10	Design Principles	37
10.1	Limitations	37
10.2	Technologies	37
10.3	Deployment Topology	37
11	Build Process	39
11.1	Steps	40
11.2	Disk Layout	41
12	Boot Sequence	43
12.1	Steps	45
13	Components	47
13.1	Custom FHS Environment	47
13.2	Android Build Environment	47
13.3	YubiKey Authentication	47
13.4	Unattended Mode	47
13.5	Remote Attestation (Keylime)	47
13.6	Credential Storage	48
13.7	Fatal Error Handling	49
13.8	Debug Mode	49

10. Design Principles

A reference implementation and proof-of-concept for offline, SLSA-compliant AOSP builds. The system is modular — individual components (attestation, FHS environment, credential storage) can be adapted or replaced for specific distribution needs.

Core properties:

- **Portable** — Runs on arbitrary `x86_64` hardware with UEFI, ≥ 250 GB disk, ≥ 64 GB memory.
- **Offline** — Requires no network beyond internal source and artifact repositories.
- **Ephemeral** — Each boot produces a pristine environment; no trace remains after shutdown.
- **Declarative** — All aspects are described in Nix expressions, ensuring identical behavior regardless of build environment or time.
- **Trusted** — Disk images are cryptographically signed. TPM-based attestation verifies the running system.

10.1 Limitations

- **aarch64 support** — only `x86_64` with UEFI is implemented.
- **Artifact uploads** — build artifacts can be persisted to a second disk but are not automatically uploaded to remote storage.
- **Configuration** — adapting to custom AOSP distributions requires familiarity with NixOS modules. See Chapter 14..

10.2 Technologies

- **NixOS / nixpkgs** — declarative system configuration, reproducible builds
- **systemd** — service orchestration, `systemd-repart` for disk images, `systemd-creds` for TPM-encrypted credentials
- **LUKS + dm-verity** — disk encryption (ephemeral + TPM2-bound) and read-only store integrity
- **erofs** — compressed read-only filesystem for the immutable store partition
- **Keylime** — TPM-based remote attestation (Rust agent, Python registrar/verifier)
- **YubiKey / PAM** — hardware-token authentication via `pam_u2f`
- **USBGuard** — USB device blocking during unattended builds
- **QEMU** — VM-based testing

An SBOM for the builder closure can be generated:

```
nix run github:tiiuae/sbomnix#sbomnix -- .#nixosConfigurations.nixos.toplevel
```

10.3 Deployment Topology

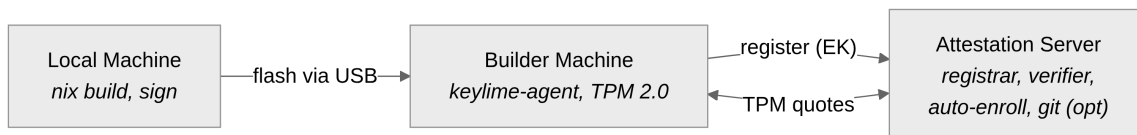


Figure 10.1: Deployment topology

- The **local machine** builds and signs disk images (requires only Nix).
- The **attestation server** runs keylime services, optionally via system-manager on non-NixOS. See Chapter 7..

- The **builder machine** boots the signed image, registers with the server, and runs AOSP builds.

11. Build Process

The disk image is assembled in multiple stages using `systemd-repart` (twice at build time, once at runtime).

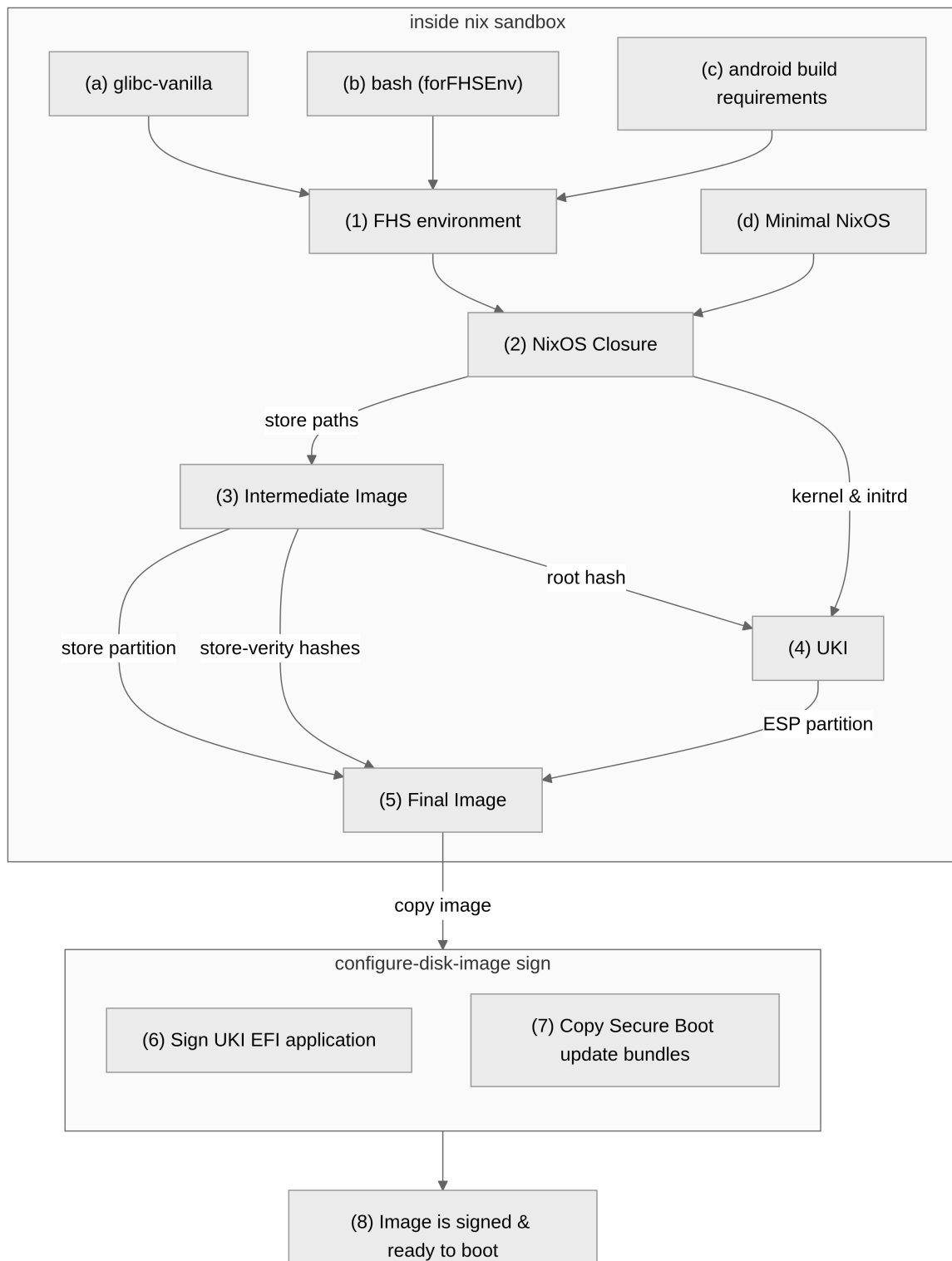


Figure 11.1: Build-time image assembly

11.1 Steps

1. **FHS environment** (Section 13.1): A derivation assembles all libraries and binaries into an FHS layout – **glibc-vanilla** (dynamic linker configured for `/lib`), **bash** (with FHS `$PATH`), and Android build dependencies.

2. **NixOS closure** (`system.build.toplevel`): The full system including boot services, the FHS derivation, and all modules.
3. **Intermediate image** (`system.build.intermediateImage`): First `systemd-repart` run. The `store` partition is populated with the NixOS closure. `boot` and `store-verity` remain empty.
4. **UKI**: dm-verity hashes are calculated from the filled store partition. A Unified Kernel Image is built with kernel, initrd, and `usrhash` on the kernel command line.
5. **Final image** (`system.build.finalImage`): Second `systemd-repart` run. The store partition is copied, verity hashes are written, and the UKI is placed on the ESP.
6. **Sign UKI**: Outside the Nix sandbox, the UKI is extracted, signed with Secure Boot keys, and written back.
7. **Secure Boot bundles**: `*.auth` files are copied to the ESP for automatic enrollment.
8. The signed image is ready to flash.

11.2 Disk Layout

Build-time (immutable):

Partition	Label	Format	Mountpoint
00-esp	<code>boot</code>	vfat	<code>/boot</code>
10-store-verity	<code>store-verity</code>	dm-verity hash	n/a
20-store	<code>store</code>	erofs	<code>/usr</code>

Runtime (created by `systemd-repart` on first boot):

Partition	Label	Format	Mountpoint	Lifecycle
31-credentials	<code>var-lib-credentials</code>	LUKS+ext4 (TPM2)	<code>/var/lib/credentials</code>	Persistent
32-keylime	<code>var-lib-keylime</code>	LUKS+ext4 (TPM2)	<code>/var/lib/keylime</code>	Persistent
40-build	<code>var-lib-build</code>	LUKS+ext4 (random key)	<code>/var/lib/build</code>	Ephemeral

The root filesystem (/) is `tmpfs` (RAM only). There is no boot loader — the UKI is loaded directly by the firmware as an EFI application.

12. Boot Sequence

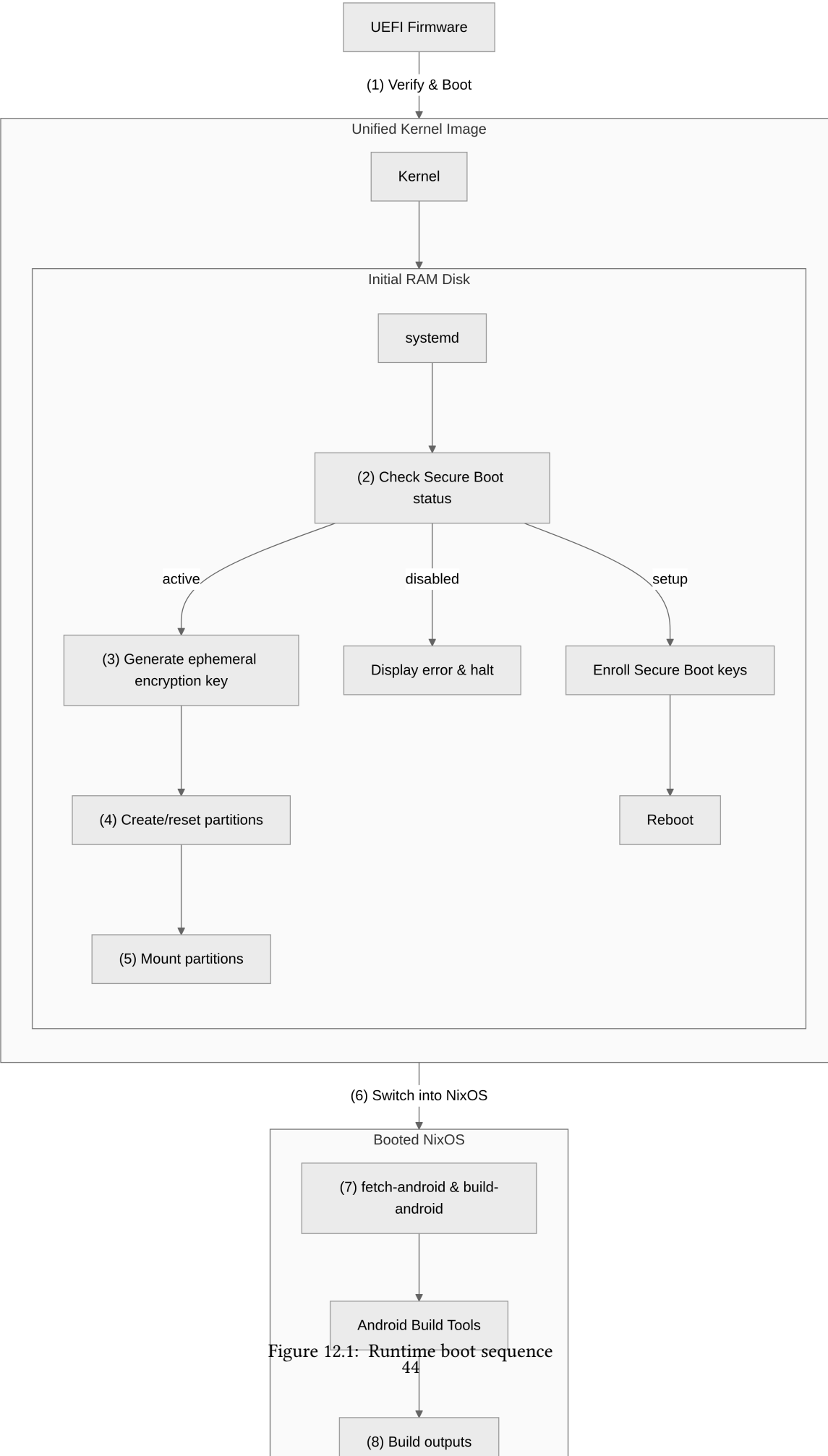


Figure 12.1: Runtime boot sequence

12.1 Steps

1. **UEFI boot:** Firmware loads the UKI, verifying its signature if Secure Boot is active.
2. **Secure Boot check** (`ensure-secure-boot-enrollment.service`):
 - **Active** → continue
 - **Setup Mode** → enroll certificates from ESP → reboot
 - **Disabled** → halt with error
3. **Ephemeral key** (`generate-disk-key.service`): 64 bytes from `/dev/urandom`, never stored on disk.
4. **Partition setup** (`systemd-repart`): On first boot, creates all mutable partitions. On subsequent boots, leaves persistent partitions untouched, factory-resets the ephemeral build partition with a fresh key.
5. **Mount:** Read-only `/usr` (dm-verity checked), bind-mounts for `/bin` and `/lib` (FHS), `tmpfs` at `/`, encrypted partitions for build data, credentials, and keylime state.
6. **Switch to NixOS:** Stage 2 boot completes.
7. **Build:** In unattended mode (Section 13.4), a configurable step sequence runs automatically. In interactive mode, `fetch-android`, `build-android`, and related tools are available at the shell (see Chapter 4.).
8. **Outputs** are in `/var/lib/build/source/out`. They are lost on shutdown unless artifact storage is enabled.

13. Components

13.1 Custom FHS Environment

A derivation bundles all declared packages into `/bin + /lib`, bind-mounted at runtime. A custom `glibc` configures the dynamic linker for `/lib`; a `bash` with `forFHSEnv` puts `/bin` in `$PATH`.

`pkgs.buildFHSEnv`, `nix-ld`, and `envfs` were tried but don't survive sandboxed bind-mounts without `/nix/store`.

13.2 Android Build Environment

The `android-build-env.nix` module adds AOSP build dependencies to the FHS environment, plus reference wrapper scripts for the default workflow:

- `fetch-android` — clone the configured repo branch
- `build-android` — set lunch target and build
- `android-sbom` — generate SBOM via upstream facilities
- `android-measure-source` — hash all files for source measurement

13.3 YubiKey Authentication

The `yubikey-auth.nix` module enforces hardware-token authentication via `pam_u2f`. Two key groups can be configured for dual-approval (two people must each touch their YubiKey). Password auth is disabled entirely.

13.4 Unattended Mode

When `nixosAndroidBuilder.unattended.enable` is set, a systemd service runs a configurable step sequence on `tty2` after boot. Steps prefixed with `root:` run with superuser privileges. Default steps:

1. `root:start-shell-if-yubikey-found` — poll for YubiKey, allow operator interrupt
2. `select-branch` — choose AOSP branch (if multiple configured)
3. `fetch-android` — clone sources
4. `build-android` — build the lunch target
5. `android-sbom` — generate SBOM
6. `android-measure-source` — hash source files
7. `copy-android-outputs` — persist build artifacts
8. `root:lock-var-lib-build` — unmount build partition, destroy encryption key
9. `root:disable-usb-guard` — stop USBGuard, re-authorize USB devices
10. `root:start-shell-and-shutdown` — interactive shell, then poweroff

USBGuard blocks USB mass storage during builds to prevent data exfiltration.

13.5 Remote Attestation (Keylime)

The Keylime agent is enabled by default on every builder image. It uses the **push model** — the agent periodically sends evidence to the verifier. Identity is derived from the TPM Endorsement Key; no pre-provisioned credentials are needed.

The agent's Attestation Key is stored on the persistent `/var/lib/keylime` partition so re-registrations after reboot use the same key.

13.5.1 Auto-Enrollment

The auto-enrollment daemon automates the agent → verifier enrollment:

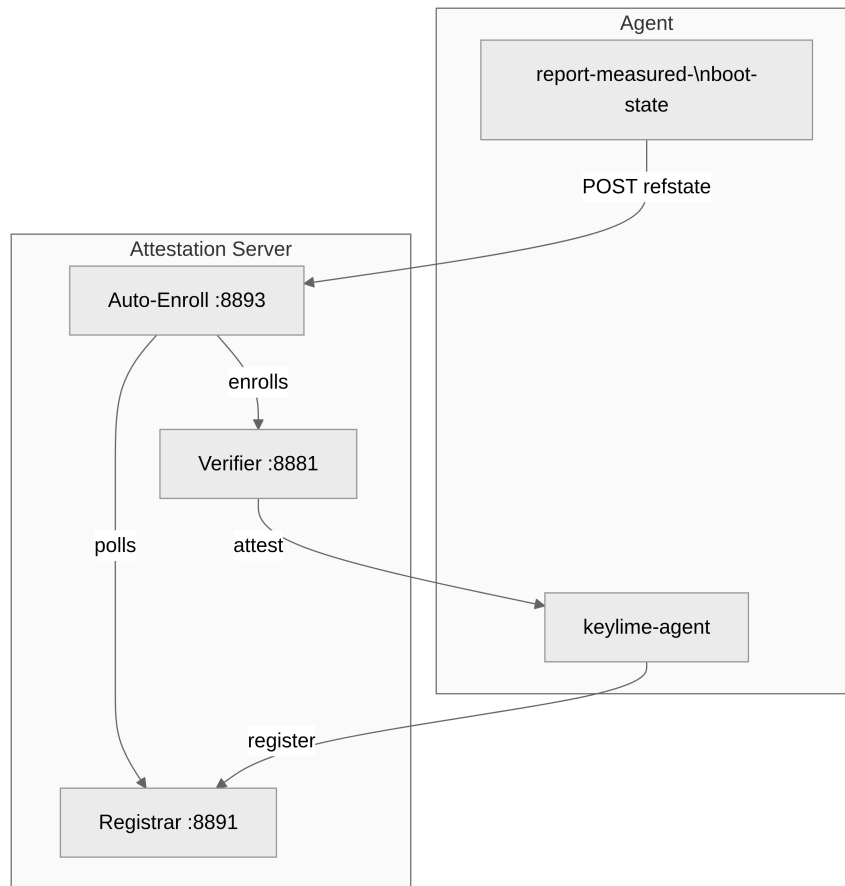


Figure 13.1: Auto-enrollment component diagram

The measured boot reference state is accepted on a **trust-on-first-use** basis — the agent self-reports before first attestation. After enrollment, the verifier replays every event log and validates every TPM quote, so false reports are caught immediately.

13.5.2 Attestation Policy

The custom `uki` policy (in `packages/keylime-measured-boot-policy/`) pins security-critical PCRs (0, 4, 7 — firmware, UKI digest, Secure Boot keys) while accepting expected variability in others. See Section 8.6 for the full validation table.

13.5.3 Tools

- `attestation-ctl` — query enrollment status, remove agents from verifier/registrar
- `measure-boot-state` — parse UEFI event log → reference state JSON
- `report-measured-boot-state` — generate and POST reference state to auto-enrollment
- `debug-measured-boot-state` — diagnose failures: replay event log, compare PCRs, diff refstates

13.6 Credential Storage

TPM-backed persistent storage via `systemd-creds`, bound to PCR 7. Credentials are stored on the `/var/lib/credentials` LUKS partition and bind-mounted to `/run/credstore.encrypted/` for automatic consumption by systemd services with `LoadCredentialEncrypted=`.

See Chapter 4. for usage of the `credential-store` command.

13.7 Fatal Error Handling

The `fatal-error.nix` module overrides systemd's emergency target with a `dialog`-based message box on `tty2`. Services write to `/run/fatal-error` on failure.

13.8 Debug Mode

`nixosAndroidBuilder.debug` adds: unauthenticated emergency shell, extra packages (vim, htop, tmux, git), nix with flakes, empty passwords, passwordless sudo, verbose logging, debug shell on `tty3`. **Not for production.**

IV

Reference

14	NixOS Options Reference	53
15	Troubleshooting	59
15.1	Secure Boot	59
15.2	Boot	59
15.3	Android Build	59
15.4	Attestation	59
15.5	VM Testing	60
16	Glossary	61

14. NixOS Options Reference

Configure the builder in `configuration.nix` (or `desktop-configuration.nix` for the desktop variant). Some options have run-time equivalents — see `fetch-android --help` and `build-android --help`.

`nixosAndroidBuilder.artifactStorage.artifactDir`

Type: absolute path

Default: `"/var/lib/artifacts"`

Directory where the persistent artifact storage is mounted.

`nixosAndroidBuilder.artifactStorage.contents`

Type: list of string

Default: [

`"*"`

]

list of files (or patterns) to copy from build outputs

`nixosAndroidBuilder.artifactStorage.diskLabel`

Type: string

Default: `"artifacts"`

disk label that identifies the storage partition

`nixosAndroidBuilder.artifactStorage.enable`

Type: boolean

Default: `false`

Whether to enable Storing outputs in an unencrypted, persistent disk partition.

`nixosAndroidBuilder.build.branches`

Type: list of string

Default: [

`"android-latest-release"`

]

List of valid branches that can be selected for building. If only one branch is specified, it will be selected automatically. If more than one branch is specified, a dialog menu will be shown for 30 seconds. First entry will be the default if no other one is selected.

`nixosAndroidBuilder.build.lunchTarget`

Type: string

Default: `"aosp_cf_x86_64_only_phone-aosp_current-eng"`

Name of lunch target to build. Can be overridden at run-time by passing `--lunch-target` to `build-android`.

nixosAndroidBuilder.build.repoManifestUrl**Type:** string**Default:** "https://android.googlesource.com/platform/manifest"URL of a `repo` manifest to fetch sources from. Can be overridden at run-time by passing `--repo-manifest-url` to `fetch-android`.**nixosAndroidBuilder.build.sourceDir****Type:** absolute path**Default:** "/var/lib/build/source"Directory where `repo` checkout is stored. Can be overridden at run-time by passing `--source-dir` to `fetch-android` and `build-android`.**nixosAndroidBuilder.build.userEmail****Type:** string**Default:** "ci@example.com"Email address used for `repo/git` operations. Can be overridden at run-time by passing `--user-email` to `fetch-android`.**nixosAndroidBuilder.build.userName****Type:** string**Default:** "CI User"User name used for `repo/git` operations. Can be overridden at run-time by passing `--user-name` to `fetch-android`.**nixosAndroidBuilder.credentialStorage.credentialDir****Type:** absolute path**Default:** "/var/lib/credentials"Directory where credentials are stored. This is the mount point of the dedicated credentials partition and will be bind-mounted to `/run/credstore.encrypted/`**nixosAndroidBuilder.credentialStorage.enable****Type:** boolean**Default:** false

Whether to enable persistent credential storage on a TPM2-bound LUKS partition.

nixosAndroidBuilder.credentialStorage.encryptionFlags**Type:** list of string**Default:** [

"--with-key=tpm2"

"--tpm2-pcrs=7"

]

Flags to pass to `systemd-creds encrypt`. See `man (1) systemd-creds`**nixosAndroidBuilder.credentialStorage.mountPoint****Type:** absolute path**Default:** "/run/credstore.encrypted"Where to mount the credential directory. `/run/credstore.encrypted/` is automatically searched by `systemd` for encrypted credentials

nixosAndroidBuilder.debug

Type: boolean

Default: false

Whether to enable image customizations for interactive access during run-time.

nixosAndroidBuilder.fhsEnv.packages

Type: list of package

Default: []

Packages needed to build Android AOSP. This is mostly copied from AOSP documentation, but could probably be reduced further, as AOSP repos ship much of it in-tree (i.e. python3, jdk, etc)

nixosAndroidBuilder.fhsEnv.pins

Type: list of package

Default: []

A sorted list of packages to add first, so that they “win” if there are collisions/conflicts during creation of the FHS env. Unresolved collisions will produce a warning in the build log.

nixosAndroidBuilder.imageId

Type: string

Default: "android-builder"

The image identifier written to /image-id on the ESP. Defaults to “android-builder” and should not normally need to be changed. Kept separate from system.name so that test VMs can override system.name (which the NixOS test driver uses as the Python machine variable name) without corrupting the image-id that configure-disk-image uses to validate image type.

nixosAndroidBuilder.unattended.enable

Type: boolean

Default: false

Whether to enable unattended mode.

nixosAndroidBuilder.unattended.steps

Type: list of string

Default: [

```
"root:start-shell-if-yubikey-found"
"select-branch"
"fetch-android"
"build-android"
"android-sbom"
"android-measure-source"
"copy-android-outputs"
"root:lock-var-lib-build"
"root:disable-usb-guard"
"root:start-shell-and-shutdown"
```

]

list of shell commands to run unattended

nixosAndroidBuilder.yubikeys.groupA

Type: list of string

Default: []

list of u2f (i.e. yubikeys) public keys for pam, as output by pamu2fcfg

pamu2fcfg -N -i “pam://nixos-android-builder” -o “pam://nixos-android-builder” -u “user”

nixosAndroidBuilder.yubikeys.groupB**Type:** list of string**Default:** []

list of u2f (i.e. yubikeys) public keys for pam, as output by pamu2fcfg

pamu2fcfg -N -i "pam://nixos-android-builder" -o "pam://nixos-android-builder" -u "user"

services.keylime-agent.enable**Type:** boolean**Default:** false

Whether to enable Keylime push model agent for TPM-based remote attestation.

services.keylime-agent.logLevel**Type:** string**Default:** "keylime_push_model_agent=info,keylime=info"

RUST_LOG filter string for the agent. For example "keylime_agent=debug,keylime=info".

services.keylime-agent.package**Type:** package**Default:** <derivation keylime-agent-0.2.9>

The keylime-agent (Rust) package to use.

services.keylime-agent.settings**Type:** attribute set of (string or signed integer or boolean or list of string)**Default:** { }Freeform settings for `agent.conf` under the `[agent]` section. Keys should use snake_case INI names matching the Rust keylime-agent config. Values are merged over built-in defaults (rust-keylime v0.2.9).Note: `registrar_ip`, `verifier_url`, and CA cert paths are configured at boot from `/boot/attestation-server.json` (written by `configure-disk-image set-attestation-server`). Build-time overrides here will be replaced at runtime if that file exists.**services.keylime.autoEnroll.enable****Type:** boolean**Default:** false

Whether to enable automatic enrollment of new Keylime agents with measured boot policy.

services.keylime.autoEnroll.enrollPort**Type:** 16 bit unsigned integer; between 0 and 65535 (both inclusive)**Default:** 8893

HTTPS port for the measured boot report endpoint.

services.keylime.autoEnroll.pollInterval**Type:** signed integer**Default:** 10

Seconds between polling the registrar for new agents.

services.keylime.enable**Type:** boolean**Default:** false

Whether to enable Keylime TPM-based remote attestation server.

services.keylime.gitServer.authPort

Type: 16 bit unsigned integer; between 0 and 65535 (both inclusive)

Default: 8895

Localhost port for the attestation auth subrequest backend. Only reachable from nginx on the same host.

services.keylime.gitServer.enable

Type: boolean

Default: false

Whether to enable attestation-gated git HTTP server.

services.keylime.gitServer.port

Type: 16 bit unsigned integer; between 0 and 65535 (both inclusive)

Default: 443

HTTPS port nginx listens on for git clone requests.

services.keylime.gitServer.repoDir

Type: absolute path

Default: "/var/lib/keylime-git/repos"

Directory containing bare git repositories served to attested agents.

services.keylime.gitServer.repos

Type: list of string

Default: []

Bare git repositories to create on first boot (without the .git suffix).

services.keylime.logLevel

Type: one of "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"

Default: "INFO"

Log level for keylime services.

services.keylime.logLevelOverrides

Type: attribute set of (one of "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL")

Default: {

"keylime.authorization.manager" = "WARNING";

"keylime.web" = "WARNING";

}

Per-logger log level overrides. Keys are Python logger qualnames (e.g. `keylime.web`), values are log levels.

By default, the noisy `keylime.web` and `keylime.authorization.manager` loggers are set to WARNING to suppress routine per-request INFO messages. Set to { } to restore the previous behaviour.

services.keylime.measuredBootPolicyPath

Type: absolute path

Default: "/nix/store/13bjblylnr3w80vksxav2np7bbhkliq0-python3.13-keylime-measured-boot-policy-0.1.0/lib/python3.13/site-packages"

Directory containing the measured boot policy Python module. Added to the verifier's PYTHON-PATH. The module must register a policy name matching `measured_boot_policy_name` in `verifier.conf`.

services.keylime.package**Type:** package**Default:** <derivation keylime-7.14.1-unstable-2026-04-02>

The keylime package to use.

services.keylime.registrar.enable**Type:** boolean**Default:** false

Whether to enable Keylime registrar service.

services.keylime.registrar.settings**Type:** attribute set of (string or signed integer or boolean or list of string or attribute set of (string or signed integer or boolean or list of string))**Default:** { }

Settings for registrar.conf [registrar] section.

services.keylime.tenant.settings**Type:** attribute set of (string or signed integer or boolean or list of string or attribute set of (string or signed integer or boolean or list of string))**Default:** { }

Settings for tenant.conf [tenant] section.

services.keylime.tls.autoGenerate**Type:** boolean**Default:** trueAutomatically generate a self-signed CA and mTLS certificates in `/var/lib/keylime/tls/` on first activation. Existing certificates are never overwritten.**services.keylime.tls.certLifetime****Type:** signed integer**Default:** 365

Validity period in days for generated certificates.

services.keylime.tls.subjectAlternativeNames**Type:** list of string**Default:** []

Additional Subject Alternative Names for the server certificate. Host IPs are auto-discovered; use this for extra DNS names or IPs.

services.keylime.verifier.enable**Type:** boolean**Default:** false

Whether to enable Keylime verifier service.

services.keylime.verifier.settings**Type:** attribute set of (string or signed integer or boolean or list of string or attribute set of (string or signed integer or boolean or list of string))**Default:** { }Settings for verifier.conf. A nested `revocations` attrset maps to the `[revocations]` INI section.

15. Troubleshooting

15.1 Secure Boot

“Secure Boot is disabled” error on boot

1. Enter firmware setup (F2/F12/Del during POST)
2. Navigate to Security → Secure Boot → Enable
3. Save and reboot

Keys won’t enroll

Ensure Setup Mode, not User Mode with existing keys:

1. Security → Secure Boot → Key Management → Clear All Keys / Reset to Setup Mode
2. Save and reboot

Boot fails after key enrollment

Re-sign with the correct keystore and verify:

```
nix run .#configure-disk-image -- status --keystore ./keys --device <image>
```

15.2 Boot

“dm-verity: Hash verification failed” — Store partition corrupted. Rebuild and reflash.

Drops to emergency shell — Check `journalctl -e`. Common causes: disk too small (needs 250 GB), hardware compatibility.

15.3 Android Build

fetch-android fails with network errors — For air-gapped environments:

```
fetch-android --repo-manifest-url=https://your-internal-mirror/platform/manifest
```

build-android fails with “command not found” — Verify FHS bind mounts:

```
mount | grep -E '/bin|/lib'
ls -la /bin/bash /lib/ld-linux-x86-64.so.2
```

Build outputs disappeared after reboot — Enable artifact storage: set `nixosAndroidBuilder.artifactStorage.enable = true`. See Chapter 4..

15.4 Attestation

Agent registers but is not enrolled (see Chapter 8. for ports and PKI) — Check `journalctl -u keylime-auto-enroll`. The daemon waits for both registration AND a measured boot report. Common causes:

- `report-measured-boot-state` service failed (check `journalctl -u report-measured-boot-state`)
- mTLS certificate issues
- Port 8893 not reachable

Agent enrolled but fails attestation — Delete enrollment and re-enroll:

```
keylime_tenant -c delete -u <agent-uuid> -v <verifier-ip> -vp 8881
```

15.5 VM Testing

“KVM not available” — Enable VT-x/AMD-V in BIOS. Verify: `lsmod | grep kvm`

Can’t exit VM — `systemctl poweroff` or close the QEMU window.

16. Glossary

AK

Attestation Key. A TPM-resident key used by the keylime agent to sign attestation quotes.

AOSP

Android Open Source Project. The publicly available source code for Android maintained by Google.

Attestation

The process of a remote party (verifier) confirming that a machine is running expected software on genuine hardware, based on TPM-signed measurements.

dm-verity

A Linux kernel feature that provides transparent integrity checking of block devices using a Merkle tree.

EFI/UEFI

Unified Extensible Firmware Interface. The modern firmware interface between the operating system and hardware, replacing legacy BIOS.

EK

Endorsement Key. A unique, manufacturer-provisioned key in the TPM that serves as the device's hardware identity. Used by the keylime agent to derive a stable UUID.

erofs

Enhanced Read-Only File System. A compressed, read-only filesystem used for the immutable store partition.

ESP

EFI System Partition. A FAT-formatted partition containing files needed to boot.

FHS

Filesystem Hierarchy Standard. A standard defining the directory structure and contents of traditional Linux systems (e.g., `/bin`, `/lib`, `/usr`).

Flake

A Nix mechanism for defining reproducible projects with locked dependencies. This project is a flake (see `flake.nix`).

initrd

Initial RAM Disk. A temporary root filesystem loaded into memory during boot, used to prepare the real root filesystem.

keylime tenant

Keylime's upstream CLI tool for enrolling and managing agents with the verifier.

Keylime

An open-source TPM-based remote attestation framework. Consists of an agent, registrar, and verifier.

KVM

Kernel-based Virtual Machine. Linux kernel module for hardware-assisted virtualization.

LUKS

Linux Unified Key Setup. The standard system for Linux disk encryption.

lunch

AOSS's build configuration selector. Chooses the target product, variant, and build type.

mTLS

Mutual TLS. A TLS connection where both client and server authenticate with certificates. Used between keylime components.

NvPCR

Non-Volatile PCR anchoring. A systemd mechanism (`systemd-tpm2-setup.service`) that extends PCR 9 at runtime, not captured in the UEFI event log.

PAM

Pluggable Authentication Modules. The Linux framework for pluggable authentication.

pam_u2f

A PAM module for U2F/FIDO2 authentication with hardware tokens such as YubiKeys.

QEMU

A machine emulator and virtualizer. Used for VM-based testing of builder and installer images.

Nix

A purely functional package manager and build system that enables reproducible, declarative builds.

NixOS

A Linux distribution built on Nix, where the entire system configuration is declared in Nix expressions.

nixpkgs

The main repository of Nix packages, containing build instructions for tens of thousands of software packages.

PCR

Platform Configuration Register. A set of SHA-256 registers inside the TPM that accumulate measurements of firmware, boot configuration, and software components. Used for attestation and credential binding.

PK/KEK/DB

Platform Key, Key Exchange Key, and Signature Database. Keys used by UEFI Secure Boot to verify boot components.

repo

Google's tool for managing multiple Git repositories, used extensively in AOSP development.

SBOM

Software Bill of Materials. A formal inventory of all components and dependencies in a piece of software.

SOPS

Secrets OPerationS. A tool for encrypting/decrypting secrets files, supporting age, GPG, and cloud KMS backends.

SCRTM

Static Root of Trust for Measurement. The initial firmware code that begins the TPM measurement chain at boot.

Secure Boot

A UEFI feature that ensures only cryptographically signed software can be booted.

Setup Mode

A Secure Boot state where custom keys can be enrolled. The firmware accepts new keys without signature verification.

SLSA

Supply-chain Levels for Software Artifacts. A tiered framework for verifying build artifact integrity and provenance.

systemd-creds

A systemd mechanism for encrypting and decrypting credentials, optionally bound to the TPM.

systemd-PCRphase

A systemd service that extends PCR 11 with boot phase strings (`sysinit`, `ready`, etc.) at runtime.

systemd-stub

The EFI stub embedded in a Unified Kernel Image. Measures UKI PE sections into PCR 11 and event tags into PCR 9.

systemd-repart

A systemd tool for declaratively creating and managing disk partitions at build time and boot time.

TOFU

Trust on First Use. A security model where the first connection is accepted without verification; subsequent connections are validated against the initial state.

tuigreet

A TUI-based login greeter used by the desktop variant.

TPM

Trusted Platform Module. A dedicated security chip that provides hardware-based cryptographic functions and key storage.

UKI

Unified Kernel Image. A single EFI executable containing the Linux kernel, initrd, and boot parameters.

system-manager

A tool by numtide that applies NixOS-style module configurations to non-NixOS Linux distributions. Used to deploy the attestation server.

USBGuard

A Linux framework for blocking unauthorized USB devices. Used during unattended builds to prevent data exfiltration.



Appendices

17	Demo Runbook: Attestation-Gated Git	67
17.1	What We're Showing	67
17.2	Pre-Demo Prep	67
17.3	Live Script	68
17.4	Recovery	68
18	Contributing	69
18.1	Quick Reference	69
18.2	Development Workflow	69
18.3	Tests	69
18.4	Project Structure	69
18.5	Security Notes	70

17. Demo Runbook: Attestation-Gated Git

Audience: demo operator (presenting to executives / customers) Format: two machines side by side (~10 min)

See Chapter 9. for the technical reference.

17.1 What We're Showing

A builder machine can only clone the Android signing-keys repository if its TPM attestation is passing. Revoking via `attestation-ctl` locks it out instantly — no reboot, no certificate rotation, no firewall change.

17.2 Pre-Demo Prep

17.2.1 1. Populate demo repository on the server

```
git init --bare /var/lib/keylime-git/repos/android-signing-keys.git
# seed with realistic content (README + dummy keys)
git -C /var/lib/keylime-git/repos/android-signing-keys.git update-server-info
ln -s /etc/keylime-git/hooks/post-receive \
    /var/lib/keylime-git/repos/android-signing-keys.git/hooks/post-receive
```

17.2.2 2. Verify agent credentials

Client cert and key are written to `/run/keylime-git/` by `report-measured-boot-state`. The `keylime-git-clone` wrapper configures git automatically.

Preflight:

```
keylime-git-clone https://<server-ip>/android-signing-keys.git /tmp/
preflight
ls /tmp/preflight/keys/
rm -rf /tmp/preflight
```

17.2.3 3. Terminal layout

Server — tmux, 3 panes:

- Top-left: ready for `attestation-ctl`
- Top-right: `journalctl -fu keylime-git-auth` (live ALLOW/DENY stream)
- Bottom: ready for `attestation-ctl remove <uuid>`

Agent — one clean pane at `~`.

17.2.4 4. Checklist (30 min before)

- ☐ Agent booted, attestation passing (`attestation-ctl status` shows PASS)
- ☐ Preflight clone succeeds
- ☐ `journalctl -fu keylime-git-auth` running
- ☐ Both screens visible

17.3 Live Script

17.3.1 Act 1 — The problem (1 min, verbal)

“This builder produces Android images signed with production keys. Those keys live in a git repo. How do you make sure only a legitimate, unmodified builder gets them?”

17.3.2 Act 2 — Attestation status (2 min)

```
attestation-ctl status
```

“The server continuously verifies this machine’s firmware, bootloader, and OS via the TPM. It passed 12 seconds ago.”

17.3.3 Act 3 — Attested clone (2 min)

On the agent:

```
keylime-git-clone https://<server-ip>/android-signing-keys.git
ls android-signing-keys/keys/
```

Server log shows: ALLOW aabbccdd-... — agent is attested

17.3.4 Act 4 — Revocation (3 min)

On the server:

```
attestation-ctl remove <uuid>
```

On the agent:

```
keylime-git-clone https://<server-ip>/android-signing-keys.git /tmp/second
# fatal: 403
```

Server log shows: DENY aabbccdd-... — agent not enrolled

“Same machine. Same certificate. Only the attestation status changed.”

17.3.5 Act 5 — Q&A (2 min)

Question	Answer
Stolen certificate?	Server checks live attestation status on every request.
Faked TPM / VM?	EK certificate verified against manufacturer CA.
How often re-checked?	Every git request; verifier re-attests every 2 seconds.
Can the builder push?	No — read-only from agent side.

17.4 Recovery

Re-enroll after demo:

```
systemctl restart keylime-auto-enroll # on server
# wait ~30 seconds
attestation-ctl status
```

18. Contributing

18.1 Quick Reference

Task	Command
Build disk image	<code>nix build .#image</code>
Run builder VM	<code>nix run .#run-vm</code>
Run desktop VM (GNOME)	<code>nix run .#run-desktop-gnome-vm</code>
Run all checks	<code>nix flake check</code>
Format code	<code>nix fmt</code>
Enter dev shell	<code>nix develop</code>
Build this manual	<code>nix run .#build-book</code>

18.2 Development Workflow

1. Edit modules in `modules/`
2. Build: `nix build .#image`
3. Test in VM: `nix run .#run-vm`
4. Run checks: `nix flake check`
5. Format: `nix fmt`

18.3 Tests

NixOS VM tests verify the build environment, Secure Boot, dm-verity, installer, and attestation:

```
nix build -L .#checks.x86_64-linux.integration
nix build -L .#checks.x86_64-linux.installer
nix build -L .#checks.x86_64-linux.keylime
nix build -L .#checks.x86_64-linux.keylime-auto-enroll
```

Tests are only re-run when inputs change. Pass `--keep-vm-state` to preserve VM state for iterative debugging.

18.4 Project Structure

```
flake.nix                # Flake definition
configuration.nix        # Builder config (site-specific)
desktop-configuration.nix # Desktop config (site-specific)

modules/                 # NixOS modules
├─ image.nix             # Disk image layout (largest module)
├─ android-build-env.nix # AOSP build toolchain
├─ fhsenv.nix            # FHS compatibility layer
├─ keylime.nix           # Keylime server
├─ keylime-agent.nix     # Keylime agent
└─ secure-boot.nix       # Secure Boot / UKI
```

```
|— credential-storage.nix    # TPM credential storage
|— yubikey-auth.nix         # YubiKey PAM
|— unattended.nix          # Unattended build pipeline
|— ...

packages/                  # Custom Nix packages
system-manager/            # Keylime server for non-NixOS
tests/                     # NixOS integration tests
docs/                      # This manual (Quarto)
```

18.5 Security Notes

- **Never commit keys** — `keys/` is gitignored
- UKI signing runs outside Nix to avoid leaking keys into `/nix/store`
- Debug mode gives unauthenticated root — production must disable it